
目次

はじめに	001
------	-----

第 NEST 章

【Rails7+esbuild】必要なJavaScript — Takahashi	001
のみ個別に読み込みたい時	

Cloudflare で独自ドメインを取得する — tanuki	002
----------------------------------	-----

未経験からフルリモートのエンジニアに — midori	003
なってもうすぐ1年なのでフルリモ向い てる人を考えてみる	

第 HUB 章

ChatGPT の効果的な使い方 — Tanaka	001
---------------------------	-----

Samplpe Title — Taro	002
----------------------	-----

第 自由章

k 近傍法でレモンサワーを探せ" — ゆーか 001

ChatGPT の効果的な使い方 — Tanaka 002

Samlpe Title — Taro 003

おわりに 004

奥付 005

はじめに

この同人誌『RUN-NEST Hub』は、技術を愛する開発者たちによる自由な寄稿によって構成されています。それぞれの筆者が日々取り組んでいるテーマや技術的な挑戦を、熱量のこもった言葉で綴っていただきました。

この誌面が、あなたの開発のヒントや新たな視点となれば幸いです。

みたいなことを未来の私が書く
よ~~~~~

Vivliostyle + Markdown ドキュメントガイド

基本的な Markdown 記法

見出し

```
# 見出し1 (h1)
## 見出し2 (h2)
### 見出し3 (h3)
```

見出し1 (h1)

見出し2 (h2)

見出し3 (h3)

段落・改行

段落は空行で区切ります。

改行は行末に半角スペースを2つ入れます。

強調

```
*斜体* または _斜体_
**太字** または __太字__
```

斜体 または 斜体

太字 または 太字

リスト

- 箇条書き
- 2つ目
 - ネストされたリスト

1. 番号付きリスト
2. 2つ目

- 箇条書き
- 2つ目
 - ネストされたリスト

1. 番号付きリスト
2. 2つ目

リンク

PDF 配布時にテキストにリンクを付けたい場合は以下のように記述します。

```
[リンクテキスト](https://example.com)
```

リンクテキスト

但し、著作物の引用などを行う場合は基本的に注釈を用いてください。

注釈

```
これは注釈付きの文章です[^1]。
```

```
[^1]: これは脚注の内容です。
```

これは注釈付きの文章です¹。

画像

画像を images フォルダに追加するか、URL リンクを取得します。

```
![説明文](../images/image.png)
```



説明文

画像の幅を指定するには次のように記述します：

```
![説明文](../images/midori.png){style="width:50%;"} 
```



説明文

コード

インラインコード：

```
これは `コード` です。
```

これは `コード` です。

コードブロック：

```
...  
console.log("Hello Vivliostyle!");  
...
```

```
console.log("Hello Vivliostyle!");
```

HTMLとの併用

Vivliostyle ではHTML も使用可能です：

改ページ

`class="page-break"` を付与する

```
<div class="page-break">ここで改行</div>
```

ここで改行

カスタムスタイルを当てたい

CSSを当てたい箇所にクラスを付与する

```
<div class="custom-box">  
  カスタムスタイルのボックス  
</div>
```

CSSを用いて装飾：

```
.custom-box {  
  border: 1px solid #ccc;  
  padding: 1em;  
  background-color: #f9f9f9;  
}
```

参考リンク

- [Vivliostyle公式サイト](#)
- [VFM仕様ドキュメント](#)
- [Vivliostyle CLI](#)

1. これは脚注の内容です。¹

【Rails7+esbuild】必要なJavaScriptのみ個別に読み込みたい時

著者名

必要なJavaScriptのみ個別に読み込みたい

方法が分からず、とりあえず動きゃいい精神でapplication.jsに全部読み込ませてたら、パフォーマンス診断で怒られてしまった（あたり前田のクラッカー）



パフォーマンス ユーザー補助



SEO



PWA



パフォーマンス

推定値のため変動する可能性があります。[パフォーマンススコアの計算](#)は、これらの指標を基に行っています。[計算ツールはこちら。](#)

▲ 0-49

■ 50-89

● 90-100

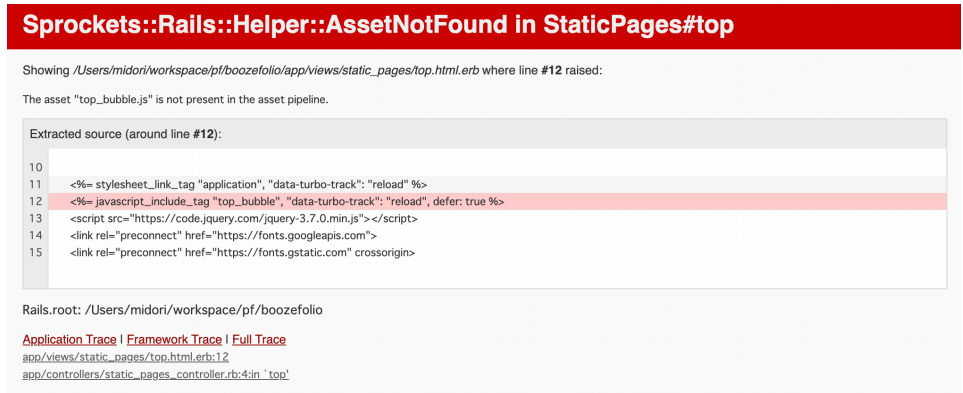


スクリーンショット 2023-08-24 10.36.39.png

不必要なjsを読み込んでいるページでブラウザエラーも出ていたので、これを機に直すことに。

そもそも、なぜ application.js に詰め込んだ？

Sprockets::Rails::Helper::AssetNotFound in ○○ のエラーになる。



スクリーンショット 2023-08-24 10.41.42.png

ここでは、top_bubble.js というファイルを読み込ませようとしています、上記エラーでうまくいきません。

The asset "top_bubble.js" is not present in the asset pipeline.

「"top_bubble.js" がアセット・パイプラインに存在しません。」
とあることから、asset 周りなのはわかるのですが、どうしたら良いのやら。。。

初学者のため、間違いやまずいやり方があればぜひコメントでご指摘ください🙏

確認したこと

1. パスが間違っていないか

2. app/assets/javascripts の設定

3. app/assets/builds に個別に読み込みたいjsが build されているか

1. と 2. は結果的には関係がなかったのですが、忘備録に簡単に記載しておきます。¹

上記記事によると Rails 6 までは app/assets/javascripts 以下の全てのファイルを自動で読み込んでいたそうです。その設定ファイルが config/initializers/assets.rb とのこと。

ただし、今回は Rails7 を使用していることから、そもそも `app/javascript` にあるのでファイル構成が違います。ものは試しなので下記のように設定を記載しましたが、結果として 3. だけで動いたため関係ありませんでした。²³

```
# Be sure to restart your server when you modify this file.

# Version of your assets, change this if you want to expire all you...
Rails.application.config.assets.version = '1.0'

# Add additional assets to the asset load path.
# Rails.application.config.assets.paths << Emoji.images_path

# Precompile additional assets.
# application.js, application.css, and all non-JS/CSS in the app/as...
# folder are already added.
# Rails.application.config.assets.precompile += %w( admin.js admin....

Rails.application.config.assets.precompile += %w( custom/*.js )
#追加してみたけど関係なかった
#今回個別に読み込みたいjsファイルは全てapp/javascript/custom以下に置いている...
```

`config/initializers/assets.rb` はいつ使うのか誰か教えてください🙏

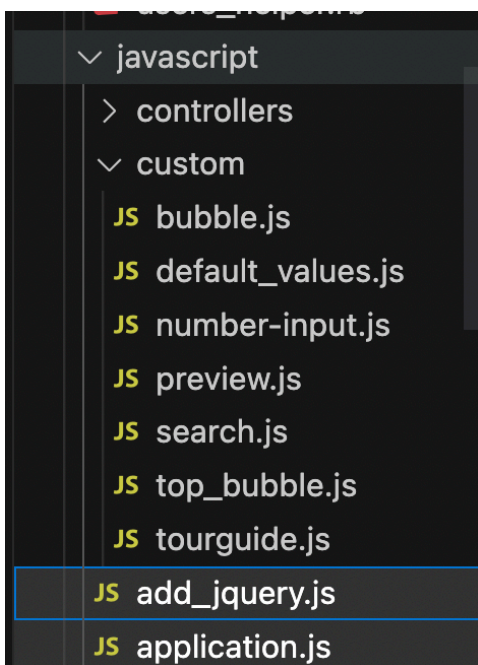
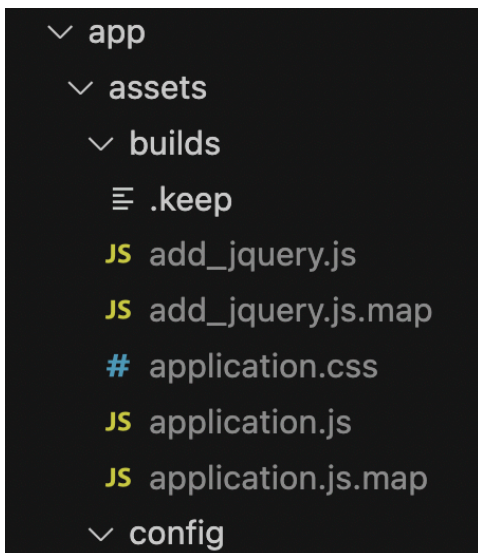
`assets.rb` についてはプリコンパイルするファイルを指定する時に必要でした。デプロイ時にエラーが出たので、こちらをご覧ください。

<https://qiita.com/MIDO-ruby7/items/9718b40c97e4d282ae83>

3. `app/assets/builds` に個別に読み込みたいjsがbuildされているか

esbuild では `app/javascript` 以下のファイルが自動で `app/assets/builds` にコンパイルされます。

※コンパイル：人間が書いたコードをコンピュータの処理装置であるCPUが解釈して実行できる形に変換すること。俺たちにとってのDeepleみたいなもんだろ（違）。

`app/javascript``app/assets/builds`

ブラウザは、この `app/assets/builds` にコンパイルされたコードを読み取るので、そもそもここにファイルがないと The asset "top_bubble.js" is not present in the asset pipeline.と言われるわけです。

esbuild の場合、どこにコンパイルのためのコマンドがあるかというと、`package.json`

```
{
  "name": "app",
  "private": "true",
  "dependencies": {
    "@hotwired/stimulus": "^3.2.1",
    "esbuild": "^0.17.19",
    省略
    "tailwindcss": "^3.3.2",
    "trix": "^2.0.5"
  },
  "scripts": {
    "build": "esbuild app/javascript/*.js --bundle --sourcemap --out...
// ↑ここでapp/javascript/*.js をassets/builds にコンパイルするよう要求して...
    "build:css": "tailwindcss -i ./app/assets/stylesheets/applicati...
  }
}
```

ということでコードを追加。

```
"scripts": {
  "build": "esbuild app/javascript/*.js --bundle --sourcemap --out...
  "build:css": "tailwindcss -i ./app/assets/stylesheets/applicati...
  "build:custom": "esbuild app/javascript/custom/*.js --bundle --...
// ↑これを追加
}
```

`$ bin/dev` コマンドで再起動したのですが、`yarn` コマンド (npm 使ってる人は npm) を打たないと自動では build されないようです。

```
$ yarn build:custom
```

```
boozefolio % yarn build:custom
yarn run v1.22.19
$ esbuild app/javascript/custom/*.js --bundle --sourcemap --outdir=...

app/assets/builds/bubble.js          254.5kb
app/assets/builds/top_bubble.js      254.4kb
app/assets/builds/tourguide.js       2.4kb
app/assets/builds/search.js          945b
app/assets/builds/default_values.js  927b
app/assets/builds/preview.js         610b
app/assets/builds/number-input.js    390b
app/assets/builds/bubble.js.map      466.1kb
app/assets/builds/top_bubble.js.map  466.1kb
app/assets/builds/tourguide.js.map   3.1kb
...and 4 more output files...

🌟 Done in 0.23s.
```

app/assets/builds に無事コンパイルされました。

```

  ▼ app
    ▼ assets
      ▼ builds
        ≡ .keep
        JS add_jquery.js
        JS add_jquery.js.map
        # application.css
        JS application.js
        JS application.js.map
        JS bubble.js
        JS bubble.js.map
        JS default_values.js
        JS default_values.js.map
        JS number-input.js
        JS number-input.js.map
        JS preview.js
        JS preview.js.map
        JS search.js
        JS search.js.map
        JS top_bubble.js
        JS top_bubble.js.map
        JS tourguide.js
        JS tourguide.js.map

```

これを読み込みたいviewファイルで読み込んでやればOK

例 : `<%= javascript_include_tag "top_bubble" %>`

その他詰まったところ

jQueryなどを使用している際は、個別にimportする必要があります。

```

import "../add_jquery" //コードを追加

var items = 30;

for (var i = 0; i <= items; i++) {
  var moveVal = Math.ceil(Math.random() * 50);
  var posVal = Math.ceil(Math.random() * 50);

```

ただ、なぜかjQueryをCDNで読み込まないとimportできず。。。application.jsでは大丈夫だったのになぜなのでしょう？ むむむ。

```
<%= javascript_include_tag "top_bubble" %>
<script src="https://code.jquery.com/jquery-3.7.0.min.js"></script>
// CDNを追加
```

このままではデプロイ時にエラーが出ました。

詳しくは別記事にまとめたので下記をご覧ください。 [4](#)

<https://keruuweb.com/rails->

[%E5%BF%85%E8%A6%81%E3%81%AAjavascript%E3%81%AE%E3%81%BF%E3%](https://keruuweb.com/rails-%E5%BF%85%E8%A6%81%E3%81%AAjavascript%E3%81%AE%E3%81%BF%E3%82%92%E8%AA%AD%E3%81%BF%E8%BE%BC%E3%82%80/)

1. [82%92%E8%AA%AD%E3%81%BF%E8%BE%BC%E3%82%80/](https://keruuweb.com/rails-82%92%E8%AA%AD%E3%81%BF%E8%BE%BC%E3%82%80/)

[↩](#)

[https://railsguides.jp/asset_pipeline.html#%E5%88%A5%E3%81%AE%E3%83%A9](https://railsguides.jp/asset_pipeline.html#%E5%88%A5%E3%81%AE%E3%83%A9%E3%82%A4%E3%83%96%E3%83%A9%E3%83%AA%E3%82%92%E4%BD%BF)

[%E3%82%A4%E3%83%96%E3%83%A9%E3%83%AA%E3%82%92%E4%BD%BF](https://railsguides.jp/asset_pipeline.html#%E5%88%A5%E3%81%AE%E3%83%A9%E3%82%A4%E3%83%96%E3%83%A9%E3%83%AA%E3%82%92%E4%BD%BF)

2. [%E3%81%86](https://railsguides.jp/asset_pipeline.html#%E5%88%A5%E3%81%AE%E3%83%A9%E3%82%A4%E3%83%96%E3%83%A9%E3%83%AA%E3%82%92%E4%BD%BF%E3%81%86)

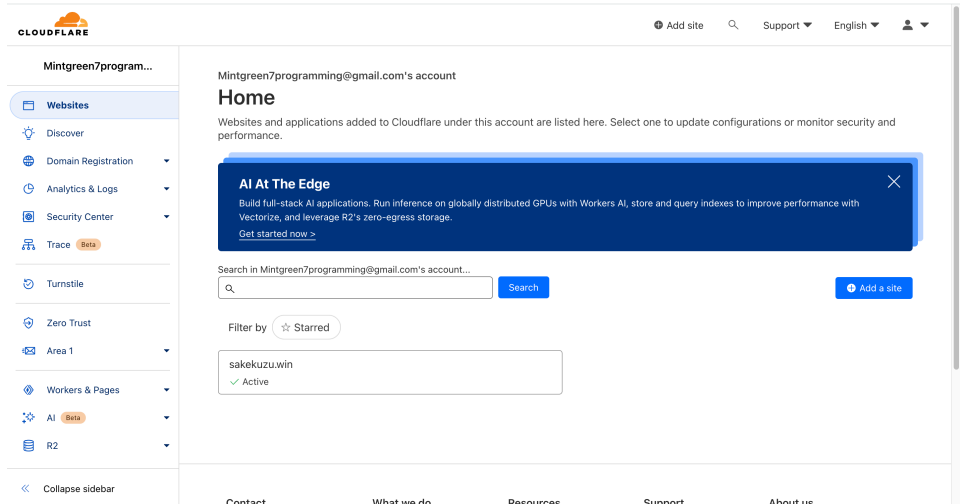
[↩](#)

3. [https://www.wantedly.com/companies/wantedly/post_articles/354873↩](https://www.wantedly.com/companies/wantedly/post_articles/354873)

Cloudflareで独自ドメインを取得する

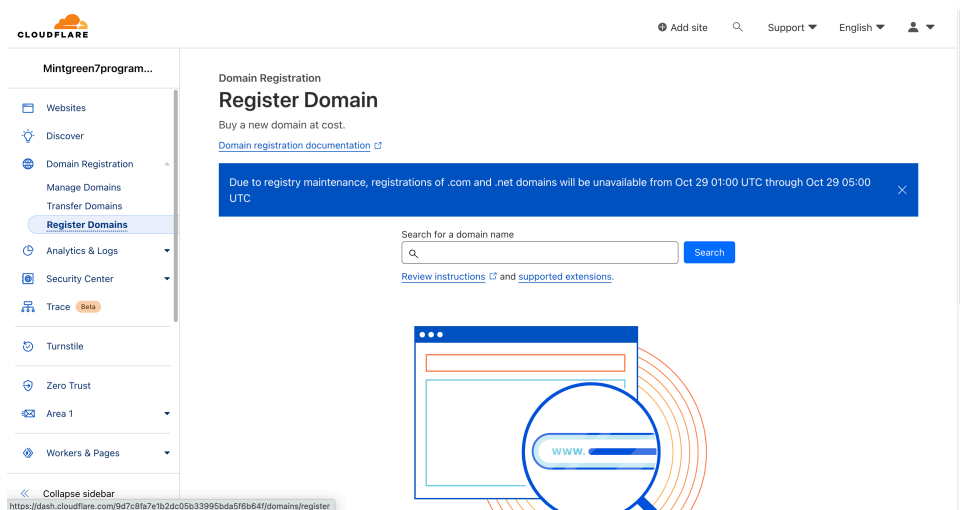
①Cloudflare に登録・ログイン

右上のプルダウンから日本語にも変えられます。



②Domain Registration > Register Domain

左のメニューから Register Domain を選択し、



🔍検索バーから、取得したいドメイン名を検索

Mintgreen7program...

- Websites
- Discover
- Domain Registration
 - Manage Domains
 - Transfer Domains
 - Register Domains**
- Analytics & Logs
- Security Center
- Trace Beta
- Turnstile
- Zero Trust
- Area 1
- Workers & Pages
- Collapse sidebar

Register Domain

Buy a new domain at cost.

[Domain registration documentation](#)

Due to registry maintenance, registrations of .com and .net domains will be unavailable from Oct 29 01:00 UTC through Oct 29 05:00 UTC

Search for a domain name

Q. kanzennirikaishita

Search

[Review instructions](#) and [supported extensions](#).

Suggested domain names

Domain	Price	
kanzennirikaishita.io	\$36.00	Purchase
kanzennirikaishitaonline.com	\$9.77	Purchase
kanzennirikaishita.net	\$10.10	Purchase
kanzennirikaishita.uk	\$4.71	Purchase
kanzennirikaishita.org	\$10.11	Purchase
kanzennirikaishita.co	\$22.00	Purchase

スクリーンショット 2023-10-21 21.13.03.png

③取得したいドメイン名を選択し、フォームに情報を入力して購入

Mintgreen7program...

- Websites
- Discover
- Domain Registration
 - Manage Domains
 - Transfer Domains
 - Register Domains**
- Analytics & Logs
- Security Center
- Trace Beta
- Turnstile
- Zero Trust
- Area 1
- Workers & Pages
- Collapse sidebar

Due to registry maintenance, registrations of .com and .net domains will be unavailable from Oct 29 01:00 UTC through Oct 29 05:00 UTC

[← Back to find a domain](#)

Complete your registration

[kanzen-ni-rikaishita.com](#)

Payment option

1 year / \$9.77

Your domain will be registered for 1 year and will expire on Oct 21, 2024.

Note: You cannot change the nameservers of a domain registered with Cloudflare. [Learn about registering a domain](#).

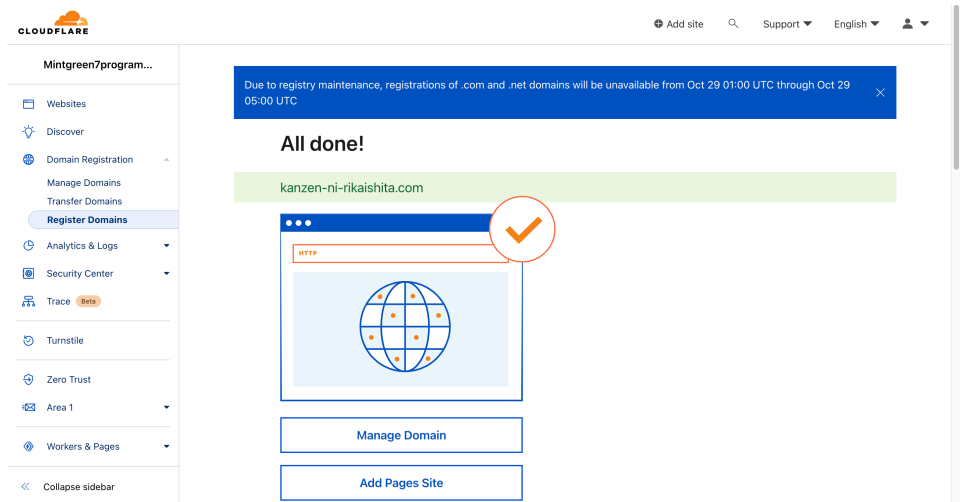
Registrant information

This is for your domain's legal record, and to contact you if there are problems with your registration.

Cloudflare will redact all personal information sent to this domain's WHOIS record.

First name *

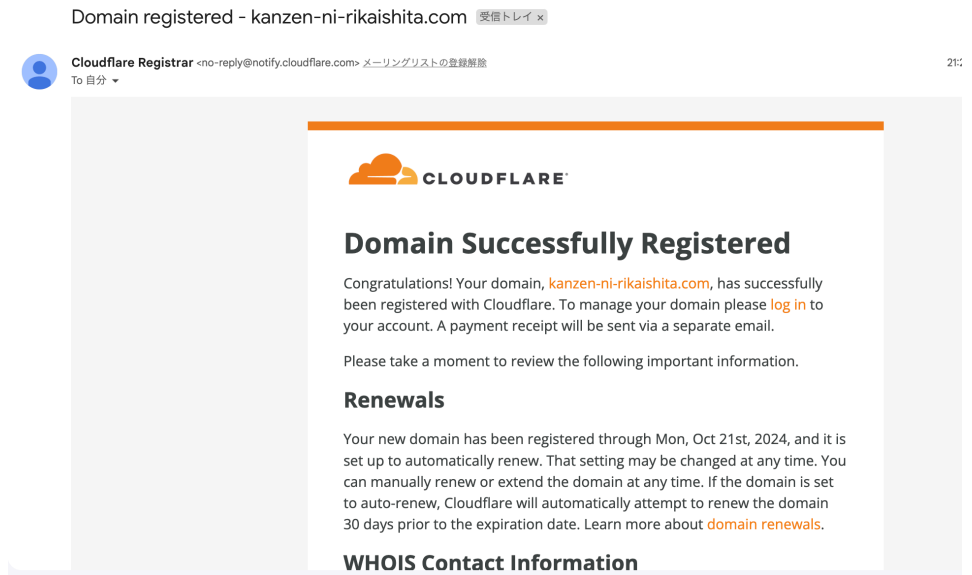
スクリーンショット 2023-10-21 21.22.35.png



スクリーンショット 2023-10-21 21.23.12.png

楽天カードはなぜか使えません🙄

ドメイン取得が完了したら以下のようなメールが来る



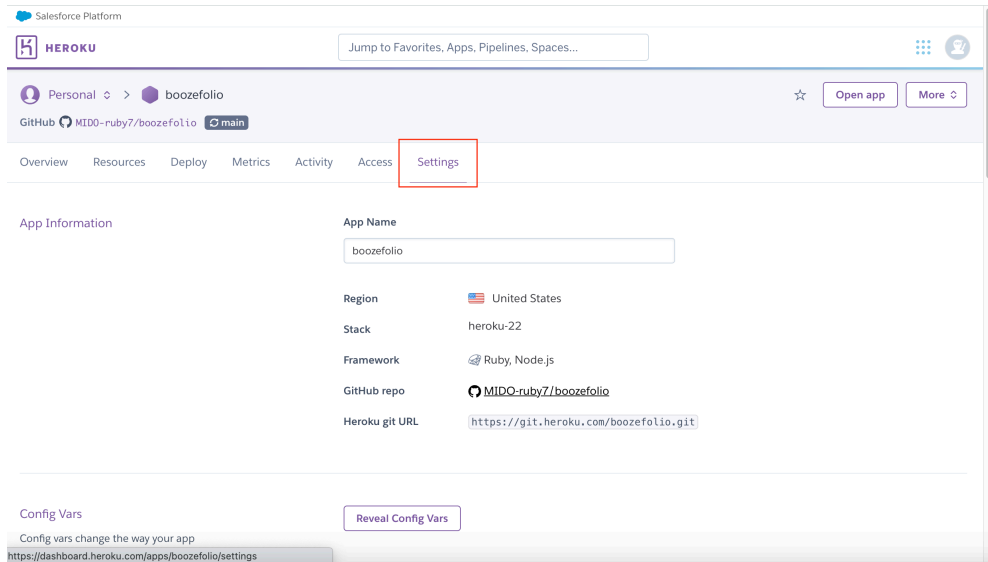
スクリーンショット 2023-10-21 21.40.35.png

ドメインが自動更新に設定されている場合、Cloudflareは有効期限の30日前に自動的にドメインが更新される

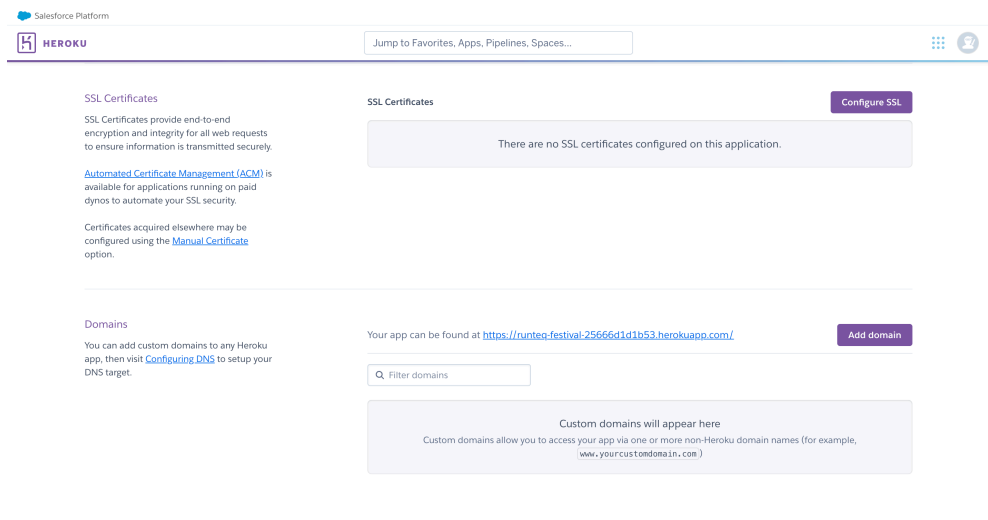
取得した独自ドメインをherokuで設定する

①heroku側の設定を行う

独自ドメインを設定したいアプリのsetting画面に遷移する

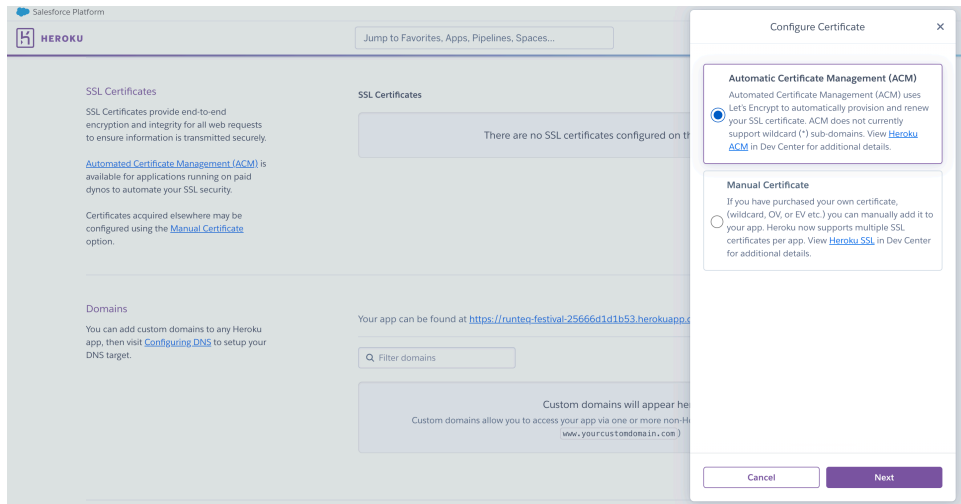


下にスクロールすると、"SSL Certificates"と"Domains"がありこの2つを設定していく



SSL Certificatesを設定する

まず、「Configure SSL」ボタンを押し、「Automatic Certificate Management (ACM)」を選択して「Next」を押す。



スクリーンショット 2023-10-21 21.52.44.png

ACMは、TLS/SSL サーバ証明書を heroku 側で自動で自動的に管理してくれる設定。

【補足：SSL サーバ証明書って？】

SSL サーバ証明書を Web サイトに設定することによって、「なりすましの防止」、やり取りされるデータの「改ざん検知」、「盗聴防止」、「中間者攻撃による通信の改ざんを検知」といった効果が得られます。アクセスしているページに SSL サーバ証明書の設定がされており、SSL によって暗号化保護がされているかを確認する手段は、ブラウザのアドレスが「https://」となっているかどうかで判断できます。

https://www.jipdec.or.jp/library/report/ssl_cert.html

"Your certificate is automatically managed. Add a custom domain to your app."と出ていればOK

SSL Certificates

SSL Certificates provide end-to-end encryption and integrity for all web requests to ensure information is transmitted securely.

[Automated Certificate Management \(ACM\)](#) is available for applications running on paid dynos to automate your SSL security.

Certificates acquired elsewhere may be configured using the [Manual Certificate](#) option.

SSL Certificates

[Configure SSL](#)

Your certificate is automatically managed. Add a custom domain to your app.

Domains

You can add custom domains to any Heroku app, then visit [Configuring DNS](#) to setup your DNS target.

Your app can be found at <https://runteq-festival-25666d1d1b53.herokuapp.com/>

[Add domain](#)[All Domains](#)[Refresh ACM Status](#)

Custom domains will appear here

Custom domains allow you to access your app via one or more non-Heroku domain names (for example, [www.yourcustomdomain.com](#))

スクリーンショット 2023-10-21 21.53.01.png

Domainを設定する。

「Add domain」を選択し、取得したドメインを入力して「Next」
(例：kanzen-ni-rikaishita.com)

SSL Certificates

SSL Certificates provide end-to-end encryption and integrity for all web requests to ensure information is transmitted securely.

[Automated Certificate Management \(ACM\)](#) is available for applications running on paid dynos to automate your SSL security.

Certificates acquired elsewhere may be configured using the [Manual Certificate](#) option.

SSL Certificates

[Configure SSL](#)

Your certificate is automatically managed. Add a custom domain to your app.

Domains

You can add custom domains to any Heroku app, then visit [Configuring DNS](#) to setup your DNS target.

Your app can be found at <https://runteq-festival-25666d1d1b53.herokuapp.com/>

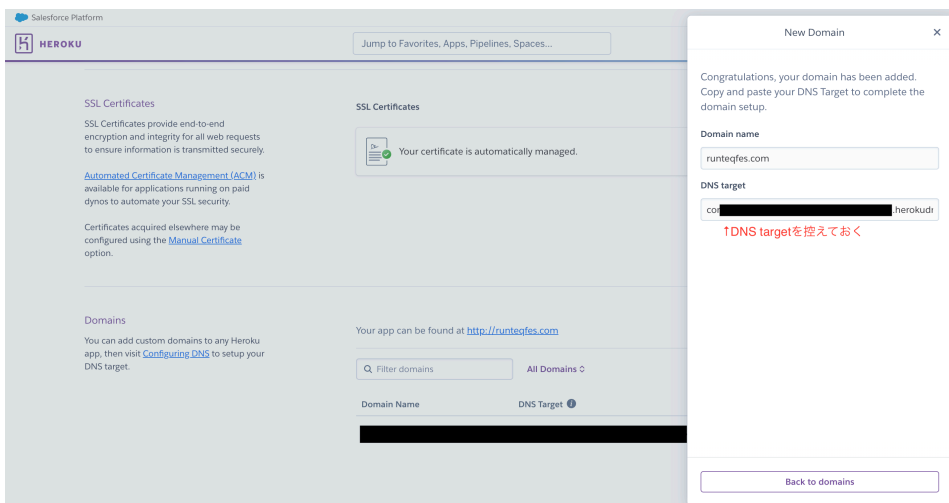
[Add domain](#)[All Domains](#)[Refresh ACM Status](#)

Custom domains will appear here

Custom domains allow you to access your app via one or more non-Heroku domain names (for example, [www.yourcustomdomain.com](#))

スクリーンショット 2023-10-21 21.53.01.png

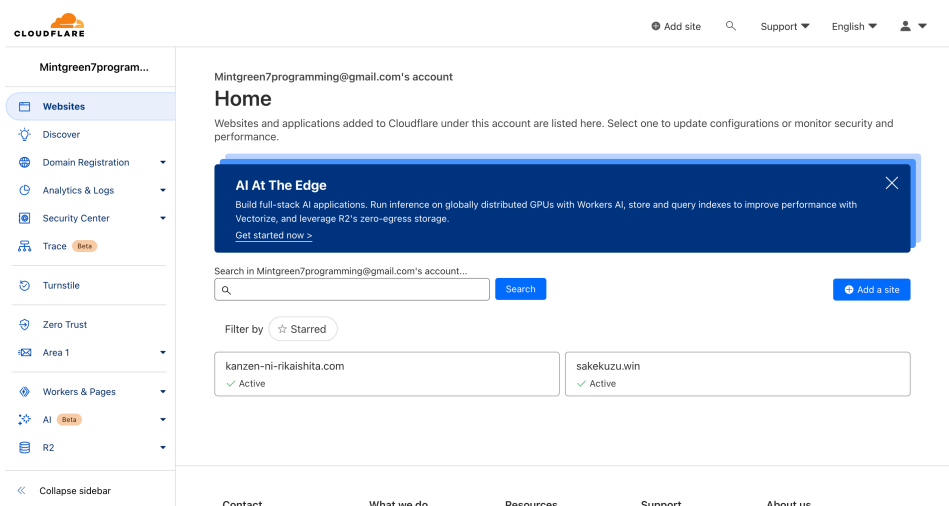
この画面が出てくるため"DNS target"の URL を控えておく。



サブドメインが必要であれば、さらに「Add domain」で追加する。
(例： www.kanzen-ni-rikaishita.com)

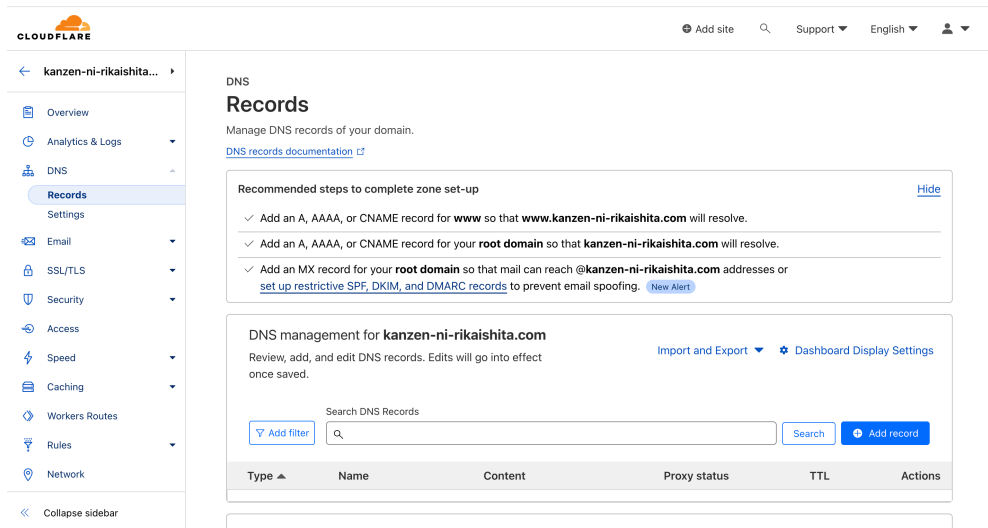
②Cloudflareで独自ドメインとherokuの発行したURLを紐付ける

CloudflareのHome(websites)から、設定したい独自ドメイン(ここでは、kanzen-ni-rikaishita.com)を選択する。

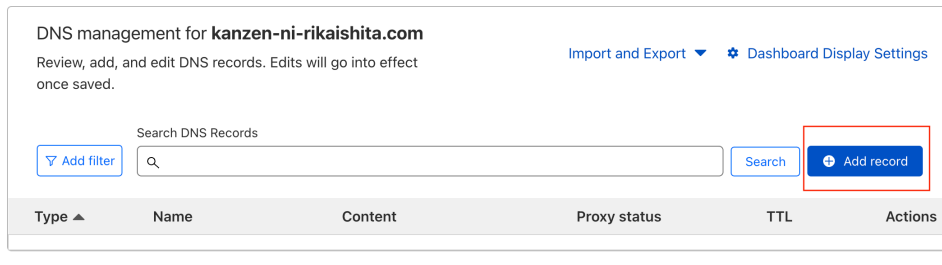


スクリーンショット 2023-10-21 21:56:09.png

DNS > Recordsを選択する



「Add record」を押す



●Typeに「CNAME」

CloudFlareではrootもサブドメインもCNAME

<https://devcenter.heroku.com/ja/articles/custom-domains#configuring-dns-for-root-domains>

●Nameにサブドメインを入力。

(サブドメインなし(root)の場合は「@」。ここでは「www」を入力し、<https://kanzen-ni-rikaishita.com>でも<https://www.kanzen-ni-rikaishita.com>両方でアクセスできるように2つ登録)

●targetにherokuのDNS targetを入力し「save」

DNS management for **kanzen-ni-rikaishita.com**

Review, add, and edit DNS records. Edits will go into effect once saved. [Import and Export](#) [Dashboard Display Settings](#)

Search DNS Records

[Add filter](#) [Search](#) [Add record](#)

www.kanzen-ni-rikaishita.com is an alias of **concave-garden-ejvtiyhegi19139x1pxq8l4m.herokudns.com** and has its traffic proxied through Cloudflare.

Type	Name (required)	Target (required)	Proxy status	TTL
CNAME	www	concave-garden-ejvtiyhegi19139x1pxq8l4m.herokudns.com	☒ Proxied	Auto

Use @ for root **herokuのDomainで控えておいたDNS targetをtargetに入力**

Record Attributes [Documentation](#)

The information provided here will not impact DNS record resolution and is only meant for your reference.

Comment

スクリーンショット 2023-10-21 22.00.22.png

Proxy status については理解が浅いため間違いがあればご指摘ください。

onにすると Cloudflare のグローバルネットワークを利用し、これによりキャッシュの最適化やより強固なセキュリティ保護を受けることができますようになりますが、DNSで登録されていたオリジンのIPアドレスではなく、Cloudflare の IP アドレスが返ってきます。

はてなブログなど IP アドレスの情報と紐づける場合は DNS only にしなければいけないようです。heroku は Proxied で設定できたのでこのままにしています。

参考：

<https://dev.classmethod.jp/articles/cloudflare-dns-basics-what-is-proxy/>

<https://blog-dry.com/entry/2023/07/12/071049>

参考：CNAME

<https://e-words.jp/w/CNAME%E3%83%AC%E3%82%B3%E3%83%BC%E3%83%89.html>

<https://www.cloudflare.com/ja-jp/learning/dns/dns-records/dns-cname-record/>

この状態ではまだ heroku 側に設定が反映されていません。

※撮り忘れたので別のアプリの写真を使用しています。すみません

Domains

You can add custom domains to any Heroku app, then visit [Configuring DNS](#) to setup your DNS target.

You have multiple custom domains enabled

Add domain

Domain Name	DNS Target	ACM Status
sakekuzu.win		Waiting
www.sakekuzu.win		Ok

アプリのターミナルで `$ heroku domains` と打ってください。

```
> Warning: heroku update available from 8.4.2 to 8.6.0.
=== boozefolio Heroku Domain

boozefolio.herokuapp.com

=== boozefolio Custom Domains
```

Domain Name	DNS Record Type	DNS Target	...
sakekuzu.win	ALIAS or ANAME	aaaaaaaaaaaaaaaaafb.herokuapp.com	a...
www.sakekuzu.win	CNAME	aaaaaaaaaaaaaaaaafb.ns.com	ab...

Domain Name が設定されていることを確認したら、

`$ heroku domains:wait` ドメイン名

例

```
boozefolio % heroku domains:wait sakekuzu.win
> Warning: heroku update available from 8.4.2 to 8.6.0.
Waiting for sakekuzu.win... done
```

Domains

You can add custom domains to any Heroku app, then visit [Configuring DNS](#) to setup your DNS target.

You have multiple custom domains enabled

Add domain

Domain Name	DNS Target	ACM Status
sakekuzu.win		Ok
www.sakekuzu.win		Ok

スクリーンショット 2023-10-23 11.50.26.png

これで完了です。

ところで独自ドメイン is 何？

よく分からないまま取得してしまっていますが、改めて独自ドメインが何なのかも少しまとめてみたいと思います。

独自ドメインとは

- ・インターネット上のアプリの住所。サーバーの位置情報を示す IP アドレスを変換した文字列の部分。
- ・ <https://runteg.jp> の場合、runteg.jp の部分がドメイン。ここが自分独自のもの、ということ。
- ・ 独自ドメインは戸建てを買うイメージ。自分だけ使える、自分のもの。一方で heroku など取得した URL は共有ドメインで、賃貸に入居してるイメージ。

<https://www.onamae.com/column/domain/50/>

https://www.value-domain.com/media/only_domain/

独自ドメインを取得すると何がいいの？

- ・ URL が変わらない
- 共有ドメインの場合、サービス会社の終了・変更によってドメイン名が変更される可能性があります。
- ・ 信頼面
- ドメイン名に会社やお店の名前、サイト名を含ませることで認知度や信頼感を高めることができます。(詐欺サイトでこの面を利用されている、という側面もありますが...)
- ・ SEO 効果がある
 - ・ メールアドレスにも使える
- などがあります。

あとポートフォリオ的には必須とのご意見も

ドメイン・HTTPS 対応

この辺りから実践的な内容。

独自ドメインと HTTPS の対応は必須です。サーバは heroku でもいいですが、<https://t.co/7oVyybP5Oq> はポートフォリオ感丸出しなので絶対 NG です。独自ドメインを設定する際に DNS を正しく理解しておくとその後の作業や運用もスムーズとなります。

続く

ー ひさじゅ@Web 系転職に強いプログラミングスクール RUNTEQ (@hisaju01)
[June 9, 2020](#)

https://www.value-domain.com/media/only_domain/

<https://muumuu-domain.com/supports/about-domain>

どこで取得できるの？

<https://jyn.jp/compare-domain-registrar/>

上記のブログ、忖度なしで書いてあり大変参考になりました。

- ・ お名前.com(GMO)
- ・ スタードメイン(ネットオウル)
- ・ ゴンベエドメイン(インターリンク)
- ・ JPDirect(JPRS)
- ・ Google Domains(Google) ⚠️現在サービス終了
- ・ Cloudflare Registrar(Cloudflare)
- ・ AWS Route53(Amazon)
- ・ Gandi.net (Gandi)

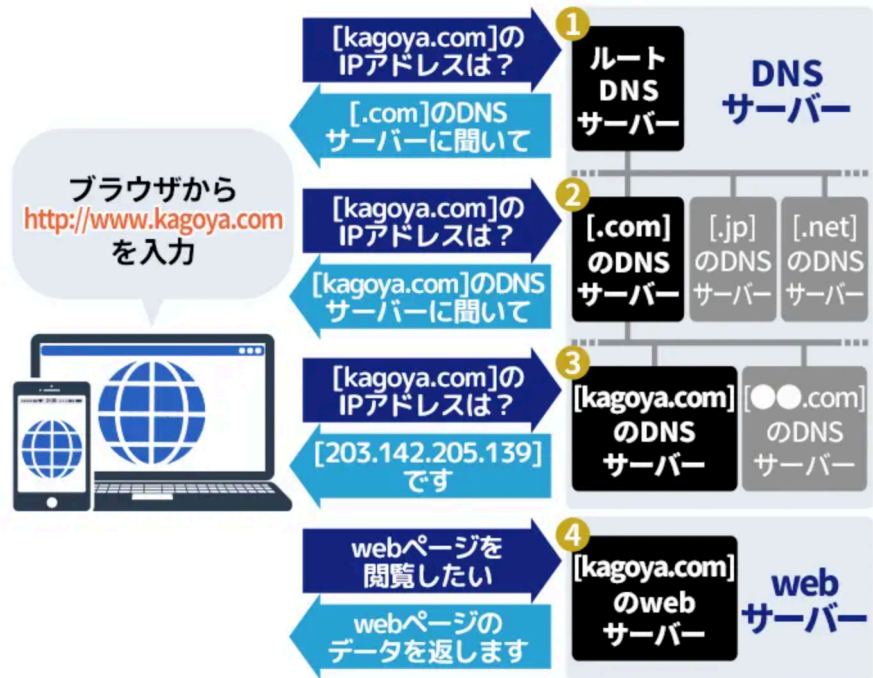
の比較が載っています。

今回は、安全性を重視かつ個人サービスなので安さも求めたいという理由で Cloudflare を使用してみることにしました。

独自ドメインを取得する上で知っておくべきこと

1. ブラウザはIPアドレスに対してHTTPリクエストを行っている

ブラウザがURLをリクエストすると、以下の図のようにレスポンスを返します。



スクリーンショット 2023-10-23 13.12.30.png

- ①PCがDNSキャッシュサーバーに、ドメイン名でアクセスする
- ②DNSキャッシュサーバーがDNSルートサーバーに、管理情報ドメイン名に対応するIPアドレスは何かを問い合わせる
- ③DNSルートサーバーがDNSキャッシュサーバーに、管理情報のあるDNSサーバーを知らせる
- ④DNSキャッシュサーバーが管理情報のあるDNSサーバーに、ドメイン名に対応するIPアドレスを問い合わせる
- ⑤管理情報のあるDNSサーバーはDNSキャッシュサーバーに、IPアドレスを回答する

- ⑥DNSキャッシュサーバーはPCに、IPアドレスを回答する
- ⑦PCはIPアドレスでWebサイトにアクセスし、ブラウザ上にWebサイトが表示される

<https://www.kagoya.jp/howto/it-glossary/domain/web-01/>

<https://www.kagoya.jp/howto/it-glossary/domain/dns-server/> より引用

サブドメインに対するクエリの場合、さらに追加のネームサーバー(DNSサーバー)が追加され、そのネームサーバーはサブドメインのCNAMEレコードの保存します。

2. 独自ドメインにかかる費用は取得費だけでない

上記のように、独自ドメインを取得することはただの文字列の売買でなく、ドメインをネームサーバーに登録する必要があります。また、取得するだけでなく所有者の登録、情報の更新など管理も必要です。

Cloudflareなどはドメインレジストラと呼ばれ、このドメインとサーバーの設定や管理を行い、ドメイン名を購入・登録できるサービスを提供している組織です。当然、そのサービスや料金体制にはそれぞれ違いがあります。

Cloudflareの場合、この登録・更新費・管理費なども全て込みの年額で変動がないため分かりやすいのも良かったです。

他社では「初年度の登録費は無料だけど次年度からは更新に数千円かかる」「Whois登録は申し込まないとやってくれない」「ドメインの更新が自動でなく、期限を過ぎたら手数料が発生した」など色々と異なるので注意が必要です。

<https://qiita.com/minorun365/items/790635f211a14ad6b4db>

<https://domain-talk.net/domain-column/comb-whois/>

3. Whois 情報に個人情報が表示される

独自ドメインは戸建てを買うイメージと言いましたが、家を買うと登記するように、独自ドメインも誰がそのドメインを持っているかなどの情報が登録・公開されます。

検索結果

kanzen-ni-rikaishita.com

```
Domain Name: KANZEN-NI-RIKAISHITA.COM
Registry Domain ID: 2823456826_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.cloudflare.com
Registrar URL: https://www.cloudflare.com
Updated Date: 2023-10-21T12:23:04Z
Creation Date: 2023-10-21T12:23:01Z
Registrar Registration Expiration Date: 2024-10-21T12:23:01Z
Registrar: Cloudflare, Inc.
Registrar IANA ID: 1910
Domain Status: clienttransferprohibited https://icann.org/epp#clienttransferprohibited
Domain Status: addperiod https://icann.org/epp#addperiod
Registry Registrant ID:
Registrant Name: DATA REDACTED
Registrant Organization: DATA REDACTED
Registrant Street: DATA REDACTED
Registrant City: DATA REDACTED
Registrant State/Province: Kagawa
Registrant Postal Code: DATA REDACTED
Registrant Country: JP
Registrant Phone: DATA REDACTED
Registrant Phone Ext: DATA REDACTED
Registrant Fax: DATA REDACTED
Registrant Fax Ext: DATA REDACTED
Registrant Email: https://domaincontact.cloudflare.com/kanzen-ni-rikaishita.com
Registry Admin ID:
Admin Name: DATA REDACTED
Admin Organization: DATA REDACTED
Admin Street: DATA REDACTED
Admin City: DATA REDACTED
Admin State/Province: DATA REDACTED
Admin Postal Code: DATA REDACTED
Admin Country: DATA REDACTED
Admin Phone: DATA REDACTED
Admin Phone Ext: DATA REDACTED
Admin Fax: DATA REDACTED
Admin Fax Ext: DATA REDACTED
Admin Email: https://domaincontact.cloudflare.com/kanzen-ni-rikaishita.com
```

(香川に住んでるのバレルヤン)

お名前.comなどのサービスでは「Whois 情報公開代理」を代行するかどうか選択する必要がありますが、これを選択すると、whois 情報に表示される個人や会社の情報の代わりに、独自ドメイン取得に使ったドメイン会社の情報が表示されるようになります。Cloudflare の場合デフォルトで Whois 情報公開代理が付属しています。

ただし、ドメインの種類によっては登録者や連絡先などの公開が必須の場合もあります。

公開されているメールアドレスにドメインに関する警告や更新を煽るような内容のフィッシング詐欺メールが届くこともあるそうなので、自分の情報が公開されているかどうか知っておく必要があるかもしれません。

4.ドメインには保有期限があり、使用実績のあるドメインは、他のサイトからのリンク設定や検索エンジンの評価が残っている

<https://www.cybersecurity.metro.tokyo.lg.jp/security/guidebook/286/index.html>

何気なく取得したドメインが実は最近まで詐欺サイトとして使われていた、という可能性もあります。独自ドメインを手放したのちに、そういったリスクもあることを知っていないと思わぬトラブルに繋がりそうです。

5.SSL化で通信を暗号化する必要がある

独自ドメインを設定しただけでは、「このページは安全ではありません」等の警告がブラウザに表示されます。これは、HTTP接続のままだからです。HTTPは平文で情報をやり取りするため、暗証番号やクレジットカード番号などが第三者に傍受される可能性があります。SSLはデータの暗号化を行う仕組みで、これを防ぐものです。

簡単なサンプルアプリなら必要ないかもしれませんが、ログイン機能などがある場合はSSL化が必須かと思います。

=====

勉強していて私があまり意識できていなかったと感じた部分は以上です。

他にも注意点があればぜひコメント等で教えてください🙏

Cloudflareで取得した独自ドメインをHerokuで設定する方法を振り返る

ここまで書いてきて、改めてCloudflareで取得した独自ドメインをHerokuで設定するとき、何の作業を行っていたのか振り返りたいと思います(長い🤔)

- ・前提として、Herokuでデプロイしたアプリは、Herokuの共有ドメインが割り当てられる([app name].herokuapp.com)。つまり、herokuapp.com および herokudns.comのネームサーバーへリクエストしてIPアドレスを探してWEBページを表示させている

- ・Herokuのsettingで「Domain」を追加すると「DNS target」が発行されネームサーバーに登録される。DNS targetは通常、ドメイン名を別のドメイン名に関連付けるCNAME(Canonical Name)レコードの値として使われる。このCNAMEレコードは、Herokuのアプリにアクセスするための一意の識別子となる

- ・ DNS target を使用して Cloudflare で独自ドメインと Heroku ドメインと関連付ける
- ・ `$ heroku domains:wait` で関連付けを反映させる
- ・ 「SSL Certificates」 で SSL を設定する。

独自ドメイン、完全に理解した🤔

その他参考サイト

<https://blog.rmblankslash.net/entry/2021/09/21/080000>

https://lpi.or.jp/lpic_all/linux/network/network07.shtml

未経験からフルリモートのエンジニアになってもうすぐ1年なのでフルリモ向いてる人を考えてみる

おかげさまで、未経験からエンジニアに転職してもうすぐ1年が経ちます。
今年お世話になった方々、改めて有難うございました。

<https://qiita.com/MIDO-ruby7/items/18ff8d14878a868b2005>

さてさて、私は転職当初からフルリモートで働いています。
フルリモで働いていることを伝えと、転職活動中の方や、内定先がフルリモートに決まった未経験エンジニアから「やっていけるでしょうか」「どんな人が向いてますか」と質問をもらうことも多いです。

そこで、今回は良く頂く質問に対して個人的見解をまとめてみようと思います。
つまりポエムです。

未経験者がフルリモートでエンジニアをやるのは可能ですか？

可能です。なぜなら、私も実際に未経験からフルリモートでエンジニアをやれているからです。

また、コロナ禍に就職されたスクールの先輩方は、皆さんフルリモートでエンジニア人生をスタートされています。コロナ禍終了後、入社されている方もいれば、そのままフルリモートで地方から仕事をされている方も多いです。

そういった方々にお話を伺うと、もちろん先輩に質問するタイミングが難しい、などの声はありましたが、コロナどうこうは関係なく、今から未経験で入るとしても同じように働けるだろうと言うお声が多かったです。

可能かどうかだけで言えば、可能です。

未経験者がフルリモートでエンジニアをやるのはやめた方がいいですか？

これは、2つの観点があります。

1. あなたが、フルリモートに向いている人か

2. あなたの所属する企業が、フルリモートに向いているか

私が企業について語るには、根拠や知識の足りない話になりそうなので、今回はどんな人がフルリモートに向いているかを考えてみたいと思います。

1.フルリモートに向いている人

報連相ができる

当たり前だと言われるかもしれませんが、実際にフルリモで働いていて、これが絶対条件だと個人的には思います。

出社する場合でも関係なく必要な能力ではありますが、他人の目が無いリモートでこれが無い場合、マネジメントやメンターの介入が遅れます。また、リモートはコミュニケーションの機会もリアルより大幅に減るため、一度すれ違いが起こると大きな問題やチーム内に支障が起こりやすいです。

例えば弊社の場合、業務について報告と相談をするのは朝10時から30分程度のミーティングだけです。全員のissueの進捗状況や困っていることが無いかを確認します。

それ以外は基本的に一人で実装を進めます。

さて、その間に困ったこと・詰まっていることがあったとき、あなたはどんな行動を取るでしょうか。

もしくは、自分の実装によるインシデントを発見したときはどうでしょうか。

自分ではなく、メンバーが困っていたらどうでしょうか。

YouTubeが見たくなったときはどうでしょうか。

他のメンバーはどうしているのでしょうか。

何を誰に報連相すれば良いのでしょうか。

周りを見渡しても誰もいません。唯一、SlackやGoogleカレンダーでメンバーの今日の予定を察することができます。

あなたは家に一人のとき、どんな行動を取って、何を優先するでしょうか。

.....なんだか、不安になりませんでしたか？

今あなたが感じた「なんだか不安」という不安感は、働き始めたあなたに対してのメンバーやマネージャーの最初の評価です。

この不安感の原因には、業務についてよく知らない、自分の力量が分からない、などさまざまな理由があります。しかし、一番根本的な原因となる問題は、あなたとメンバーがお互いについてを良く知らないことです。

未経験から転職して働き始めたとき、あなたやメンバーが**どんな人で、何を考えて、どう行動するのか**を理解することが、最初のハードルになります。

リモートは、この相手との信頼関係を築くコミュニケーションの機会が圧倒的に少ないです。人間は、顔を見て今日の天気の話をしただけでも相手への緊張感は下がり理解度が上がります。

「報連相ができる」と一言で言いましたが、リモートの場合は、この機会を自ら作り、適切にコミュニケーションや会話をする必要があるのです。自分の考えをきちんと伝え、素直に困ったこと・嬉しかったこと・やりたいことなどを開示し、自分からコミュニケーションを取れる人が「報連相ができる」人です。

「サポートは必要だが、この人なら放っておいても安心」とメンバーから信頼される努力ができない人は、リモートに向いてないと思います。

新卒がフルリモートに向かないのは、この周囲の安心感を得るのがどうしても最初は難しいこと、仕事上のやり取りが身につく前にフルリモートをする事で失敗を相談できなかったり孤立するからです。

逆に言えば、新卒でもバイトやインターン、研究室、部活などの経験からこれが備わっている人は最初からフルリモートできるのではないかなと思います。

前職が営業だった経験から、転職など新しい集団に入った最初の1ヶ月にどんなことすれば良いのかをまとめた記事があります(タイトルはRUNTEQについてですがその中で"組織で必要とされる人材になる行動指針"について考えた記事です)。

お互いについてを良く知らないことがなぜハードルになるのか、具体的にどんな行動をすれば良いのか参考に知りたい方は覗いてみてください。

https://note.com/mintgreen_mido/n/n7a78982b9840#5256EBCD-5FCC-4742-8BA6-29661839B18B

自分で自分をコントロールできる

これも、社会人として必要な力かと思いますがフルリモートの場合はもっと切実に健康面でこれが必要だと実感しています。

一日中家にいるので、自分の健康状態・精神状態を把握しづらいのです。

人と話していてイライラする

今日はなんだか歩くと息切れする

前向きな気持ちになれない

日常の中で、些細な身体サインがあるかと思いますが外に出たり人と接する時間が少ないのでこのサインを見過ぎがちです。

また、外に出ないことで、だんだんと慢性的な疲労感や疲れを感じ始めます。やばい。

さらに言えば、外に出ていればスタバでマックしたり、マクドでマックしたり、色々と気分転換の機会がありますが、フルリモートだとその機会も意識的に取らないといけません。

- 自分がどんな状態の時に疲れているのか知っている
- 自分が疲れている時の気分転換の方法を知っている

かどうかは非常に重要だと思いました。多分、これが下手だと長期的に病むか・死にます。私は身の危険を感じております。

余談

最後にする話ではないのですが、フルリモートで働いてみて見つけた健康面のメリットは**自由に気兼ねなくトイレに行けるのでお通じが良くなったこと**です。便秘の人にはおすすめです。共感してくれる人がいると嬉しいです。

フルリモ向いてる要素は、他にも色々あると思いますが、私が実感しているフルリモート1年目の感想は、特に上記2点かなと思います。

皆さんが思う、フルリモートに向いている資質があればぜひコメントで教えてください。

k 近傍法でレモンサワーを探せ

はじめに

「レモンサワーに似た飲み物は何だろう？」

居酒屋でレモンサワーを頼んで飲むと...ん？レモンサワー？いや、何か別の飲み物に似ているような...

居酒屋飲み放題あるある、「これお酒入ってる？」「レモンの味...する？」...というありふれた日常から生まれたこの疑問を、機械学習を使って解決してみました。

とはいえ私の機械学習の知識はほぼゼロのレベルです。毎週実施している輪読会で **k 近傍法**を知り、レコメンド機能ってこういうアルゴリズムからできるんだ！と興味を持ち今回の実装に挑戦してみました。エンジニアが集まるとつい、「どれ頼んだらレモンサワーの味するドリンクくるのかな？」と**デバッグ感覚で飲み比べを始めてしまう**。そんなくだらないからこそ楽しい記憶をふと思い出し、k 近傍法で検索してみることにしました。

自己紹介

1996 年、愛知県生まれの元薬剤師。2024 年 12 月よりエンジニア転職し、現在は Java メイン。プログラミングスクールで Ruby、実務で PHP を経験し、言語にこだわらず課題解決の手段として積極的に学習中です。

趣味は読書と愛犬の柴犬と戯れること、バドミントンなどなど。

ゴール

Ruby 学習者の視点から「**くだらない疑問を真面目に解決する**」姿勢で筆者のレベルで、下記を比較・考察します。

- 両言語の処理速度や実装の違い
- その結果「レモンサワーに似た飲み物」は何になるのか

※検索に使用したデータはダミーデータのため検索結果はあくまで参考です。

なぜこの比較をしたのか

機械学習という Python のイメージが強いですが、Ruby にも **Annoy ライブラリ** などが存在することを知りました。

しかし Python と比較すると Ruby は機械学習に関する情報が少ない印象もあり、実際の性能や開発体験はどうなのか疑問でした。

また、単純な技術比較ではなく、この技術を使ってどのように課題を解決できるのかという観点も含めて検証したいと考えました。

データの構造

ドリンクの基本情報

drink_id,name,category,abv,ingredients

- category: sour, cocktail, coffee, beer 等
- abv: アルコール度数
- ingredients: 材料(ベクトル化対象)

1. 開発効率の比較

まずは開発にかかった時間とコード行数を比較し考察します。

開発時間の内訳比較

工程	Python(40 行)	Ruby(120 行)	差(倍)
環境構築	30 分	30 分	1.0 倍
データ前処理	60 分	60 分	1.0 倍
アルゴリズム実装	120 分	240 分	2.0 倍
テスト・デバッグ	60 分	60 分	1.0 倍
合計	270 分	390 分	1.4 倍

【Python：ライブラリの豊富さが開発を加速】

scikit-learn の使用でドキュメントも破りやすく、わずか数行で高性能な類似度検索を実現できました。

【Ruby：開発効率の高さでストレスのない実装】

Annoy は Python と同様にライブラリを使用しましたが、それ以外は python の実装を参考に精度を少し落とし、自前実装したためコード量は増えましたが Ruby の豊富な標準メソッドやメソッドチェーンにより、実装はストレスなく進められました。

2. 実際の実装を紹介

ドリンクの材料をベクトル化し、k 近傍法による検索を実装していきます。Python と Ruby 共に C 言語で作成された Annoy を使用し、Python では scikit-learn、Ruby では自前実装でベクトル化とコサイン類似度計算を行いました。

ベクトル化

まずは比較対象となるドリンクの材料を比較できるようにベクトル化していきます。

【Python (scikit-learn)】

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(min_df=1, max_df=0.7, analyzer='char',...
tfidf_matrix = vectorizer.fit_transform(ingredients_texts)
```

ライブラリがあるため基本的に **TfidfVectorizer クラス** のドキュメントを参考に実装しました。ただし、sklearn (パッケージ名: scikit-learn) は非常に大きなライブラリであったため使用するクラスのみインポートして使用しました。

文字レベルの N-gram の採用理由：

peach_juice と peach_liqueur のように、異なる単語でも共通の文字列 (peach) を持つ場合の類似性を考慮できるよう採用

【Ruby(単語頻度ベクトルを自前実装)】

```
def vectorize_drinks(drinks_data)
  all_ingredients = drinks_data.flat_map { |drink| drink['ingredient...

  drink_vectors = {}
  drinks_data.each do |drink|
    # vector: 材料が使われた回数をcount
    vector = all_ingredients.map { |ingredient| drink['ingredients'...
    # drink_vectors: ハッシュにベクトル(出現回数)を蓄積
    drink_vectors[drink['drink_id']] = {
      name: drink['name'],
      category: drink['category'],
      vector: vector
    }
  end

  drink_vectors
end
```

Python では **TF-IDF** というベクトル化手法を使用しましたが、Ruby には同様のものがなく自前実装は厳しいと判断したため**単語頻度ベクトル**によるベクトル化を実装しました。

選定理由は比較的小規模で構造化されたデータを扱っているため（ドリンクの材料）、TF-IDF のように単語の重みを下げ精度を上げる手法を実装せず十分な結果が得られると考えました。

使用した Ruby の標準メソッド抜粋

- `flat_map` - 配列の各要素にブロックを適用し、結果を平坦化した新しい配列(全ての材料が格納)を作成

【補足：TF-IDF と単語頻度ベクトル】

TF-IDF や単語頻度ベクトルは単語や文章をベクトル化するための手法です。TF-IDF は単語の重みを下げるため同義語や類似概念が増えてくるとより効果を発揮します。例えば下記の 4 つのドリンクがある場合...

- ユーザー A/桃ジュース(ピーチ果汁)
- ユーザー B/ピーチウーロン(ピーチリキュール)
- ユーザー C/ピーチオレンジ(ピーチリキュール)
- ユーザー D/ピーチソーダ(ピーチリキュール)

単語頻度ベクトルではユーザー A のみマッチングされない可能性が発生します。

なぜかという単語頻度ベクトルの場合、「ピーチリキュール」と「ピーチ果実」の関連は認識がなく、類似度が低くなるからです。

一方 TF-IDF では、「ピーチ」に対して単語の重みを高くするため類似度が大幅に向上し、マッチングされやすくなります。

コサイン類似度計算

続いてベクトル化した材料の類似度を計算できるようにしていきます。

【Python (cosine_similarity)】

```
from sklearn.metrics.pairwise import cosine_similarity

# ベクトル化した行列(tfidf_matrix)の全てのベクトル同士の類似度を計算
similarities = cosine_similarity(tfidf_matrix)
```

こちら基本的にはドキュメントを参考に実装しました。

【Ruby (自前実装)】

```
def cosine_similarity(vec1, vec2)
  # 内積を計算
  dot_product = vec1.zip(vec2).map { |a, b| a * b }.sum
  # ベクトルの大きさを計算
  magnitude1 = Math.sqrt(vec1.map { |x| x**2 }.sum)
  magnitude2 = Math.sqrt(vec2.map { |x| x**2 }.sum)

  # 材料データの不足時に安全策として0.0を返し処理を終了する
  return 0.0 if magnitude1 == 0 || magnitude2 == 0
  # コサイン類似度を計算
  dot_product / (magnitude1 * magnitude2)
end
```

Python には `cosine_similarity` がありましたが ruby には同様のものはなく、標準ライブラリ・メソッドを使用して自前実装しました。

使用した Ruby の標準ライブラリ・メソッド抜粋

- `zip` - コサイン類似度計算で内積を求める必要があるため、要素ごと1つずつにペアとする例としては、下記のように使用している。

```
vec1 = [1, 2, 3]
vec2 = [4, 5, 6]
vec1.zip(vec2) # => [[1, 4], [2, 5], [3, 6]]
```

- `Math.sqrt` - 平方根に変換する

【補足：コサイン類似度の数式】

コサイン類似度 = $(A \cdot B) / (|A| \times |B|)$

-1~1 の範囲で 1 に近いほど類似していると言えます。

k 近傍法による検索

最後に Annoy を使用して検索の実装をしていきます。

【Python (Annoy)】

```
from annoy import AnnoyIndex

# vector_dim: ベクトルの次元数
vector_dim = tfidf_matrix.shape[1]
# 1.インデックス作成
index = AnnoyIndex(vector_dim, 'angular')
```

```
# 2.ベクトル追加
for i in range(tfidf_matrix.shape[0]):
    # Annoyライブラリを使用するため1次元に変換
    vector = tfidf_matrix[i].toarray().flatten()
    index.add_item(i, vector)
# 3.インデックス構築（検索可能な状態になる）
index.build(n_trees=10)
```

Annoy ライブラリを使用したためドキュメントを参考に実装しました。

検索までの流れは、**1.インデックス作成 2. ベクトル追加 3. インデックス構築**です。

使用した Python の標準ライブラリ抜粋

- `shape` - sklearn の内部で使用されている行列オブジェクトで配列や行列の次元を取得する役割があります
例として、下記のように使用しています。

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.shape) # (2, 3) - 2行3列
```

【Ruby (Annoy)】

```
def annoy_search(drink_vectors)
  vector_dim = drink_vectors.values.first[:vector].length
  index = Annoy::AnnoyIndex.new(n_features: vector_dim, metric: ...

  drink_ids = drink_vectors.keys
  drink_ids.each_with_index do |drink_id, i|
    vector = drink_vectors[drink_id][:vector]
    index.add_item(i, vector)
  end

  index.build(10)
end
```

Python では sklearn を使用しデータ構造が2次元の行列となっているため、行：飲み物、列：材料の TF-IDF 値となっている。

Ruby ではデータ構造がハッシュ形式となっているため、配列（[:vector]）の長さが次元数となる。

【補足：build メソッドについて】

Annoy ライブラリの近似最近傍探索インデックス構築処理をする。build 後に検索が可能となります。

3. 実行結果で比較

処理速度

Python と Ruby で一部実装内容は異なりますが、処理速度を比較しました。
比較内容としてデータ量は 3 パターン（10, 100, 1000 件）、処理範囲は 4 パターン（ベクトル化完了時間、類似度計算完了時間、Annoy 検索完了時間、総処理時間）で実施しました。

データ量による性能の変化

データ量	Python	Ruby	性能比
10 件	0.001474 秒	0.000888 秒	Ruby が 1.66 倍高速
100 件	0.001888 秒	0.048502 秒	Python が 25.7 倍高速
1,000 件	0.007339 秒	5.073059 秒	Python が 691.2 倍高速

10 件では Ruby が 1.66 倍、100 件以上では Python が圧倒的に高速という結果でした。
Ruby が高速となった理由について、データ数 10 件と 100 件の場合の結果をさらに深掘り考察していきます。

【データ量 10 件で実施した結果】

処理項目	Python 時間	Ruby 時間	性能比
ベクトル化	0.001191 秒	0.000054 秒	Ruby が 22.1 倍高速
類似度計算	0.000278 秒	0.000778 秒	Python が 2.8 倍高速
Annoy 検索	0.000005 秒	0.000056 秒	Python が 11.2 倍高速
総処理時間	0.001474 秒	0.000888 秒	Ruby が 1.66 倍高速

小規模データ（10 件）ではなぜ Ruby が高速だったのか

- ベクトル化の処理を精度を下げて単純な自前実装にしている
- ただし、類似度計算や検索は Python の最適化ライブラリが優位
- 今回の小規模データでは「ライブラリ初期化の重さ」より「シンプル実装の軽さ」が勝った

【データ量 100 件で実施した結果】

処理項目	Python 時間	Ruby 時間	性能比
ベクトル化	0.002173 秒	0.000314 秒	Ruby が 6.9 倍高速
類似度計算	0.000405 秒	0.050983 秒	Python が 125.9 倍高速
Annoy 検索	0.000010 秒	0.000563 秒	Python が 56.3 倍高速
総処理時間	0.001888 秒	0.048502 秒	Python が 25.7 倍高速

なぜ Python が高速になったのか

- ベクトル化のみ Ruby が有利だが、小規模データでは 22.1 倍高速であったのに対して 6.9 倍まで落ちており、それ以外の処理項目は Python が圧倒的
- scikit-learn や NumPy による 並列化・メモリ最適化・C 実装 の効果大きい
- 小規模データではライブラリの起動時間が不要な分、手作り実装が有利だが、100 件以上の大規模データでは最適化されたライブラリが圧倒的に有利。

4. 結果「レモンサワーに似た飲み物は何だろう？」

さて、ここまで実装方法やその速度についてみてきました。いよいよ最後にレモンサワーに似た飲み物についてみていきます。

比較する飲み物を 5 個用意し、レモンサワーに似た飲み物を検索していきます。

レモンサワーに似た飲み物の検索結果

結論としては「**レモンサワーに似た飲み物はハイボールレモン**」となりました。

2 位のレモンモヒート (Python: 0.4814、Ruby: 0.5164) もレモン系の飲み物ですが、炭酸がない点でレモンサワーとの類似度が下がっていると考えられます。さらに 3 位のレモンジュース (Python: 0.3485、Ruby: 0.3333) はアルコールが含まれていないため、さらに類似度が低くなったと考えました。

この結果から、Annoy ライブラリは飲み物の特徴（アルコールの有無、炭酸の有無、フレーバーなど）を適切に学習し、人間の直感に近い類似性を計算できていることがわかります。

順位	Python 結果	Python 類似度	Ruby 結果	Ruby 類似度	差
1	highball_lemon	0.6154	highball_lemon	0.6667	-0.0513
2	lemon_mojito	0.4814	lemon_mojito	0.5164	-0.0350
3	lemon_juice	0.3485	lemon_juice	0.3333	+0.0152
4	orange_mojito	0.2091	orange_mojito	0.2582	-0.0491
5	kahlua_milk	0.0000	kahlua_milk	0.0000	0.0000

- 最大差: 0.0513 (highball_lemon)
- 平均差: 0.0301 (3.01%)
- 上位 3 位の順位: 100% 一致

補足:ピーチサワーに似た飲み物の検索結果

先ほど「TF-IDF と単語頻度ベクトル」で補足説明した箇所を実際に行ってみました。
レモンサワーの結果と異なるポイントは、「**ピーチ果汁**」と「**ピーチリキュール**」の類似性を判断できるかです。

peach_oolong の結果に注目してください。Ruby では自前実装のため少し精度を落とし、単語頻度ベクトルでベクトル化しているため同じピーチ系のドリンクでも「ピーチ果汁」を使用する **peach_highball**, **peach_mojito**, **peach_juice** に対して、「ピーチリキュール」を使用する **peach_oolong** は類似性があると判断できず**類似度は 0.0** という結果になりました。一方、Python ではライブラリを使用し類似性があると判断できるため**類似度が 0.1606** と計算されていることがわかります。

順位	Python 結果	Python 類似度	Ruby 結果	Ruby 類似度	差
1	peach_highball	0.6564	peach_highball	0.6667	-0.0103
2	peach_mojito	0.5667	peach_mojito	0.5164	+0.0503
3	peach_juice	0.4010	peach_juice	0.3333	+0.0677
4	peach_oolong	0.1606	peach_oolong	0.0	+0.1606
5	kahlua_milk	0.0093	kahlua_milk	0.0	+0.0093

- 最大差:0.1606(peach_oolong)
- 平均差:0.0552(5.52%)
- 上位 3 位の順位:100%一致

最後に

「レモンサワーに似た飲み物は何だろう?」という単純な疑問から始まり、「まあハイボールだね」と想定通りの結果を k 近傍法を用いて導くことができました。何気ないこの疑問から技術的な探究に発展するとは思いませんでしたが、興味から入ったからこそ探究心も強く、飽きずにのめり込めたため、今後も真面目にふざける姿勢で探究心を持って学習を続けていきたいと思いました。

きっかけこそくだらない疑問でしたが、机上のアルゴリズム学習で興味を持った k 近傍法を実際に手を動かして触れることにより、データ量によって処理速度の差に違いがあったり、ベクトル化の手法にも様々あることを知ったりと多くの発見がありました。

実装では公式ドキュメントを参考に数行書くだけで実装可能な、Python のライブラリの強さを実感したと同時に、Ruby の豊富な標準メソッドの魅力も再認識できました。

特に印象的だったのは、同じアルゴリズムでもベクトル化の手法によって結果が変わることです。Python では TF-IDF で文字レベルの N-gram を採用することで、`peach_juice` と `peach_liqueur` のような微妙な違いも捉えられるようになり、想定通りの結果が出た時にはとても感動しました。

改めて、薬剤師からエンジニアへ転職して楽しいと思うのは「正確性より、まず動かしてみる」という考え方がメインになったことかもしれません。今回 k 近傍法に関して手を動かしてみるきっかけになったのは、一緒に学ぶ仲間がいたからこそでした。興味を持った内容を議論できる相手がいたから実装に挑戦し、完璧な実装ではありませんが、実際に手を動かして試行錯誤し比較することで多くの学びを得られました。

最後まで読んでくださり誠にありがとうございました！初めての挑戦で締め切りに追われながらも、最後まで真面目にふざけて楽しく書かせていただきました。

参考文献

- なっとく!アルゴリズム第2版

ChatGPT の効果的な使い方

Samlpe Title

ChatGPT の効果的な使い方

Samlpe Title

おわりに

最後までお読みいただき、ありがとうございました。

本誌の制作は、多くの寄稿者と技術コミュニティの協力によって実現しました。

感想やフィードバックは、次回の制作にも活かしていきたいと考えています。

X（旧 Twitter）で `#RUNNEST_Hub` を付けて投稿いただけると嬉しいです。

奥付